Riphah International University, Lahore

Name: Meeral Nafees

SAP ID: 59190

Course: Game Programming

Submitted to: Sir Hisham Zahid

Assignment: 03
High-Level Code Architecture Design for Multiplayer Social Deduction Games: A Case Study on Among Us

1. Game Overview

The selected title for this architectural analysis is *Among Us*, a cross-platform, asynchronous multiplayer social deduction game developed and published by the American game studio Innersloth. Released initially for mobile platforms in June 2018 and subsequently ported to Windows and major home consoles, the title operates on the Unity Engine and relies on cross-platform real-time network synchronization. The core gameplay loop centers on a group of up to 15 players who are dropped into an isolated, networked map—such as a spaceship or planetary base—and partitioned into two distinct, asymmetrical factions: the Crewmates and the Impostors.

From a mechanical perspective, Crewmates are tasked with navigating the 2D environment to complete a series of localized minigames, internally classified as tasks, which are distributed across various logical zones within the map. The objective of the Crewmate faction is to either successfully complete all globally assigned tasks, filling a shared progression metric, or successfully deduce and exile all Impostors. Conversely, the Impostor faction is tasked with covertly eliminating the Crewmates until numerical parity is achieved, utilizing map traversal shortcuts known as vents, and triggering critical system failures (sabotages) to divide the crew and manipulate their spatial positioning.

The gameplay is fundamentally bifurcated into two distinct operational phases. The primary phase is a real-time action simulation where players move across a two-dimensional plane, interacting with environmental triggers and one another. The secondary phase, known as the discussion or meeting phase, halts all real-time movement and physical simulation to facilitate a synchronized, chat-based deliberation and voting sequence. During this phase, input handling shifts entirely from spatial navigation to user interface interaction, requiring a distinct shift in the underlying state machine.

In recent iterations, the architectural complexity of the game has expanded significantly to accommodate highly specialized sub-roles that introduce unique asymmetrical mechanics, demanding more robust state synchronization and validation. The Crewmate faction features roles such as the Scientist, who queries a global variable array to monitor player vitals; the Engineer, who possesses limited, cooldown-restricted access to the Impostor's vent network; the Tracker, who can affix networked location beacons to other players to poll their coordinates; and the Guardian Angel, a state applied to deceased players allowing them to cast temporary protective shields over living entities. The Impostor faction has similarly evolved, introducing the Shapeshifter, an entity capable of temporarily overriding its visual and

identifying network components to mimic another player; the Phantom, which toggles client-side rendering flags to achieve temporary invisibility; and the Viper, a role that dynamically alters the standard death sequence by scheduling the destruction of the resultant dead body object, leaving no physical evidence.

From a technical and software engineering standpoint, *Among Us* presents a unique set of challenges and trade-offs. The architecture must facilitate real-time synchronization of player coordinates and states over highly variable network conditions, particularly considering its massive mobile player base. However, unlike a high-fidelity competitive shooter, the spatial mechanics do not require sub-millisecond precision for physics interactions, allowing for a decoupling of physical colliders from network authority. The system must seamlessly transition between decentralized, real-time spatial client prediction and a highly synchronized, authoritative state machine during the voting phases, all while maintaining strict anti-cheat postures to prevent the premature broadcast of hidden roles to unauthorized clients.

2. High-Level Architecture

The software architecture of *Among Us* is built upon the Unity Engine, utilizing C# for all game logic and leveraging the IL2CPP (Intermediate Language to C++) backend for cross-platform compilation and memory management optimization. To support a diverse ecosystem of mobile, personal computer, and console clients without incurring prohibitive infrastructure costs, the networking topology employs a hybrid Client-Host Relay model, heavily utilizing a custom Reliable User Datagram Protocol (RUDP) implementation known as Hazel.

2.1 The Client-Host Relay Topology

In standard peer-to-peer (P2P) networking paradigms, clients connect directly to one another. While this eliminates dedicated server hosting costs, it exposes client IP addresses, introduces severe Network Address Translation (NAT) punch-through complexities, and makes the session highly susceptible to host-advantage, where the host's local simulation dictates the state of all remote clients. Conversely, a fully authoritative dedicated server processes all game logic centrally, preventing most forms of cheating but drastically increasing hosting costs and development complexity, as the server must run a headless instance of the physics engine.

To circumvent the limitations of both extremes, the architecture implements a Relay Server topology. This model separates the responsibilities of data routing and state authority.

The Relay Server, representing the backend infrastructure, is hosted on lightweight cloud instances. During its peak periods, the game operated successfully on minimal hardware, such as Amazon AWS nano instances in the US East region. The server acts purely as a packet router, receiving serialized byte streams from clients and broadcasting them to the other authenticated clients within a specific logical session or lobby. Because it does not run a headless Unity instance or compute physics, its resource overhead is exceptionally low; a single server operating at three percent capacity can manage millions of data transactions, routing roughly 500 megabytes of data daily during normal operations, and scaling effortlessly during player spikes. The server performs minimal logic processing, primarily handling DTLS-encrypted connection handshakes, lobby creation, matchmaking queue management, and basic impersonation or spoofing checks to prevent unauthorized clients from transmitting

data masquerading as another user.
Within this relay ecosystem, one player in the lobby is designated as the Client-Host, or Master Client. While the Host does not route traffic, their local machine acts as the state arbiter for globally consequential events. The Host dictates when the game transitions from the matchmaking lobby to the active game state, resolves simultaneous interaction conflicts (such as two players attempting to report a body at the exact same millisecond), and manages the lifecycle of globally timed events or artificial intelligence entities. If the Host disconnects, the architecture supports host migration, sequentially promoting the next connected client to the arbiter role to maintain session continuity.
The individual Clients run full, local simulations of the game world. They rely heavily on client-side prediction, immediately rendering movement inputs locally while buffering these inputs and simultaneously transmitting their spatial coordinates to the Relay Server for peer broadcast. This approach completely masks network latency for the local user, assuming the server will implicitly validate the actions unless contradicted by a superior authority, such as the Host executing a state transition.

2.2 Network Transport Layer: The Hazel Protocol

The foundation of the high-level architecture is the Hazel Networking Library, a low-level C# framework tailored for connection-oriented, message-based communication over UDP. Standard Transmission Control Protocol (TCP) guarantees packet delivery and order but suffers from head-of-line blocking, where a single dropped packet stalls the entire data stream until acknowledged, making it wholly unsuitable for real-time spatial movement. Conversely, standard User Datagram Protocol (UDP) is fast and avoids head-of-line blocking but offers no guarantees regarding packet delivery, sequence order, or duplication, which is catastrophic for critical game events like casting a vote or executing a kill.
Hazel bridges this gap by implementing RUDP (Reliable UDP). It maintains custom internal sliding windows, sequence numbers, and acknowledgment (ACK) tracking over standard UDP sockets. This design provides the developer with multiple delivery channels tailored to the specific nature of the data payload.

Delivery Protocol	**Delivery Guarantee**	**Ordering Guarantee**	**Use Case in Architecture**
Reliable	Guaranteed. Lost packets trigger retransmission timeouts. Duplicates dropped.	Not Guaranteed.	Critical state changes: MurderPlayer, CastVote, ReportDeadBody, Role Assignments.
ReliableSequenced	Guaranteed. Duplicates	Guaranteed order.	Sequential game state transitions

	dropped.		where chronological order is strictly mandatory.
Unreliable	Not Guaranteed. Fire-and-forget delivery. Duplicates dropped.	Not Guaranteed.	High-frequency spatial telemetry via CustomNetworkTransform. Older coordinates are rendered obsolete by newer arrivals.
UnreliableSequenced	Not Guaranteed.	Guaranteed order. Older packets are discarded.	Rapid animation state synchronization.

By utilizing Hazel, the game segments its traffic effectively. Movement data is dispatched unreliably, ensuring that a dropped packet on a degraded mobile network does not pause the local simulation. When a packet is lost, the client simply ignores the gap and interpolates toward the next successfully received coordinate packet. Conversely, irreversible actions, such as an Impostor executing a kill command, are transmitted via the Reliable channel. The Hazel socket automatically tags the packet with a sequence identifier, buffers it, and resends it periodically until the receiving client issues a 0x0a Acknowledgement packet confirming receipt. Furthermore, the architecture utilizes DTLS to encrypt packets, securing the payload against man-in-the-middle packet sniffing and rudimentary memory reading.

2.3 Subsystem Breakdown

To manage the complexity of a multi-client session featuring highly interactive environments and asymmetrical player abilities, the architecture subdivides gameplay mechanics into discrete systems. These systems operate atop Unity's MonoBehaviour paradigms and are networked through a custom object synchronization framework.

2.3.1 Player System

The Player System encapsulates all client-specific data and logic. Rather than deploying a monolithic class structure to govern a player, the architecture adheres to composition, assembling player entities from modular networking components.

The primary brain of the player is the PlayerControl component. This system manages global attributes, input handling, and logical state. It tracks identifiers such as PlayerId, the user's friend code, and specific rendering customizations like color and cosmetics. More importantly,

it stores the assigned role and the binary state of life (Alive or Dead), exposing functions to execute kills, report bodies, and manage task arrays.
Spatial representation and movement logic are entirely offloaded to sibling components. The PlayerPhysics system manages the calculation of movement vectors based on local input, processes raycasts against wall colliders to prevent clipping, and tracks situational states, such as whether a player is actively navigating a vent or climbing a ladder. The CustomNetworkTransform system specifically handles the serialization and deserialization of the player's 2D spatial coordinates. Instead of broadcasting floating-point vectors continuously, this system aggressively quantizes data, utilizing linear interpolation upon receipt to smoothly translate remote characters across the local screen, masking the underlying latency inherent in the Unreliable UDP data stream.

2.3.2 Matchmaking and Lobby System

The Matchmaking System interfaces directly with the Relay servers, bypassing complex session management by utilizing a highly compressed identifier protocol. A lobby is uniquely identified internally by a standard int32 integer. However, to facilitate user-friendly sharing, this integer is presented to the client as a six-letter alphabetical code. This translation is achieved through a custom Base-26 bitwise encoding algorithm. Version 2 codes (six letters) are stored as negative integers, while legacy four-letter codes utilized basic ASCII hex equivalents. This algorithmic compression ensures that lobby handshakes consume minimal bandwidth.
Once connected to a lobby, the system manages the GameOptionsData object. This centralized configuration matrix synchronizes dozens of granular variables—including Impostor count, kill cooldown durations, movement speed multipliers, vision radius, and the specific distribution percentages of advanced roles like the Scientist or Shapeshifter. The Master Client dictates these variables, and the system guarantees their synchronization across all connected clients prior to initiating the match state.

2.3.3 Game State Management

The global state transitions are orchestrated by an overarching GameManager instance. This system dictates the primary state machine of the session, progressing linearly through predefined phases: Lobby -> Intro Sequence -> Gameplay -> Emergency Meeting -> Gameplay -> End Sequence.
The Game State Management system is completely event-driven. It passively listens for critical threshold triggers emitted by other subsystems. For example, it monitors an aggregated integer array tracking completed tasks across all Crewmates. Simultaneously, it evaluates the ratio of living Impostors to living Crewmates upon every PlayerControl death event. If the task array reaches maximum capacity, or if the numerical parity between factions is achieved, the GameManager immediately broadcasts an EndGame payload, forcefully overriding the local states of all clients and displaying the victory or defeat sequence.

2.3.4 Combat and Interaction System

The architecture decouples combat and interaction from Unity's standard physical collision matrix. Relying on network-synchronized rigidbodies for precise melee combat in a

high-latency mobile environment frequently results in desynchronization, where a strike appears valid locally but registers as a miss on the server. To resolve this, the Combat System relies purely on mathematical distance checks executed locally on the client.
When an Impostor triggers the "Kill" input, their local client calculates the Euclidean distance to the nearest valid target. If this distance falls within the bounds specified by the GameOptionsData (e.g., Short, Normal, or Long kill distance), the client bypasses physics entirely and directly dispatches a MurderPlayer Remote Procedure Call (RPC) via the Hazel Reliable channel. The Relay Server broadcasts this RPC. Upon receiving the payload, the victim's client validates the request, updates their PlayerControl to a deceased state, instantly spawns a networked DeadBody prefab at their exact current coordinates, and initiates the ghost transition logic, completely avoiding physics engine disputes.

2.3.5 Environment and Sabotage System (ShipStatus)

The game environment is not treated as static geometry; it is constructed as a highly interconnected network entity encapsulated by the ShipStatus class. This class serves as the root network registry for all map-specific interactable elements, dynamically loading localized implementations such as SkeldShipStatus, MiraShipStatus, PolusShipStatus, or AirshipStatus.
The ShipStatus contains an array of specialized subsystems governing environmental manipulation. These include the ReactorSystem, LifeSuppSystem, SecurityCameraSystem, and DoorsSystem. When an Impostor initiates a sabotage, their client sends an RPC directed at the ShipStatus network identifier. The ShipStatus subsequently routes this command to the appropriate child system. If the LifeSuppSystem is targeted, it updates its internal boolean flags, triggering localized alarms across all clients, initiating a global countdown timer, and temporarily disabling specific player interactions until another client successfully executes a RepairSystem RPC at the designated environmental coordinate.

2.3.6 UI and Voting System (MeetingHud)

The user interface architecture adheres strictly to a data-driven model, updating visual elements purely in reaction to events emitted by the underlying network objects. The most complex architectural implementation within the UI layer occurs during the discussion phase. When a player discovers a dead body and triggers an interaction, their client sends a ReportDeadBody RPC. This command forces the GameManager to transition states, which simultaneously instructs the server to instantiate a specialized, highly synchronized network object known as the MeetingHud. The MeetingHud overrides standard player input, disables the CustomNetworkTransform updates to lock physical positions, and renders the voting interface.
This object is solely responsible for orchestrating the voting matrix. It exposes specific RPC endpoints, primarily 0x18 CastVote, 0x19 ClearVote, and 0x1a AddVote. As clients make selections, the MeetingHud reliably synchronizes the state of who has voted, without revealing the specific target until the polling period concludes. Once the Master Client determines the timer has expired, the MeetingHud tallies the internal arrays, broadcasts the final result via 0x17 VotingComplete, and triggers the 0x04 Exiled RPC against the PlayerControl component of the losing entity, seamlessly transitioning the state machine back to active gameplay.

3. Code Structure

To ensure a highly maintainable, performant, and easily extensible codebase, the software architecture leverages a custom network object framework built atop Unity's Entity Component System (ECS) architecture. The foundational core of this framework is the InnerNetObject.

3.1 Classes and Interfaces

The InnerNetObject Base Class

At the root of the synchronization architecture is the InnerNetObject abstract base class. Any entity that requires state replication across the network ecosystem inherits from this class. The framework assigns every InnerNetObject a globally unique NetworkId (a packed integer) and an OwnerId (identifying the client responsible for initiating the object's lifecycle).
The InnerNetObject defines the fundamental contract for serialization and network message handling through virtual methods that derived classes must override:

- Serialize(MessageWriter writer, bool initialState): Packs the object's current local state variables into a binary stream. If initialState is true, it forces a complete synchronization; otherwise, it may only serialize delta changes.
- Deserialize(MessageReader reader, bool initialState): Unpacks the received binary stream and applies the data to the local variables.
- HandleRpc(byte callId, MessageReader reader): A crucial routing function containing a switch statement. It reads the incoming callId and routes the subsequent payload data to the appropriate local function, bridging the network layer with the application layer.

Component Responsibilities

The primary actors within the game are constructed using a composition-over-inheritance approach, assembling specific InnerNetObject scripts onto empty Unity GameObjects.

Component Class	Primary Responsibilities	Handled Remote Procedure Calls (RPCs)
PlayerControl	Governs core logic, input validation, role definition, task lists, and cosmetic assignment. Acts as the primary interface for game rules.	0x06 SetName, 0x08 SetColor, 0x09 SetHat, 0x0c MurderPlayer, 0x0b ReportDeadBody.
PlayerPhysics	Calculates movement vectors, manages colliders, and tracks specific physical	0x13 EnterVent, 0x14 ExitVent, 0x1f ClimbLadder.

	states (e.g., vent occupancy, ladder traversal).	
CustomNetworkTransform	Aggressively quantizes 2D coordinates into optimized byte structures and transmits them via Unreliable UDP. Handles linear interpolation (Lerp) for smooth rendering.	0x15 SnapTo (Instant positional override, bypassing standard interpolation routines).
NetworkedPlayerInfo	A lightweight data container object persistently storing the state of all players in a lobby. Used extensively by the UI to query player data without instantiating heavy physics objects.	Handles property synchronization for IsDead, IsDisconnected, and IsImpostor.

Global System Managers

In addition to entity components, the architecture utilizes persistent global manager objects to maintain the overarching simulation state.

- **GameData:** This object synchronizes the overarching state of the lobby and manages the array of NetworkedPlayerInfo structs. By utilizing GameData, external systems can query if a player is an Impostor (via PlayerInfo.Role.IsImpostor) or read their current cosmetic selections without requiring direct references to the physical PlayerControl GameObjects in the scene.
- **ShipStatus:** Acts as the geographical and systemic registry for the active map. It holds polymorphic arrays of objects implementing ISystemType or ITask interfaces, handling the serialization of complex map-wide states like door locks and oxygen levels.

3.2 Diagram: Architectural Object Flow

The following sequence illustrates the architectural flow of data, tracing a single state-altering event from the network socket layer, through the routing systems, into the Unity application layer, and finally to the user interface.
[Network Layer (Hazel Protocol via UDP Port 22023)] | | (Raw Byte Array representing a Reliable Packet) v [HazelUDPSocket / Network Manager]
| (Validates DTLS Encryption, Sequence IDs, and returns 0x0a Acknowledgement) | (Extracts Payload. Type: 0x05 GameData) v [GameData Router]

| (Reads Target NetworkId from payload to find the specific InnerNetObject) v
[Unity ECS / MonoBehaviour Layer]
[Target: InnerNetObject Registry] | +--> (If Target == ShipStatus NetID)
| --> Route to ShipStatus.HandleRpc() -> SabotageSystem | +--> (If Target == PlayerControl NetID [e.g., Net ID: 209])
| --> Route to PlayerControl.HandleRpc(callId, reader) | --> Case 0x0b: Execute ReportDeadBody(Target ID) | --> GameManager overrides state to 'Meeting' | --> Server spawns 'MeetingHud' Object | +--> (If Target == CustomNetworkTransform NetID) --> Deserialize Quantized Vector2 --> Initiate Lerp Coroutine
[Presentation / UI Layer] | | (Subscribes to events emitted by GameManager and MeetingHud) v [HUD Manager] --> Halts Player Input, Renders Voting Matrix

3.3 Communication Between Systems

The architecture employs a strictly decoupled, event-driven communication strategy to maintain boundaries between the network layer, the logical simulation, and the presentation layer, preventing tightly coupled code dependency loops.

3.3.1 Network Communication (Client-to-Client via Server)

When an InnerNetObject initiates a state change that requires global replication, the system dynamically constructs a byte payload using a MessageWriter. The internal structure of these packets is highly structured to minimize footprint:

1. **Nonce & Hazel Tag:** Identifies the packet format (e.g., Reliable) and the Hazel message type (e.g., 0x05 GameData).
2. **Game ID:** A 32-bit integer indicating the target lobby.
3. **RPC Tag:** Identifies the payload as a Remote Procedure Call (0x02).
4. **Target NetID:** Specifies which InnerNetObject instance should receive the call (sent as a PackedUInt32 to save bytes).
5. **Call ID:** A single byte identifying the specific function (e.g., 0x0b for ReportDeadBody).
6. **Payload Parameters:** Appended bytes for function arguments, such as the PlayerId of the victim.

This byte array is passed to the Hazel socket, which packages it and transmits it via the Reliable or Unreliable UDP channel, depending on the operational requirements of the callId.

3.3.2 Local Communication (System-to-System)

Upon receipt and deserialization of a network packet, the InnerNetObject must notify the broader Unity application layer. It accomplishes this through local C# events and delegates. For instance, when the PlayerControl object processes an 0x04 Exiled RPC, it directly mutates its internal life state. Subsequently, it invokes an OnExiled delegate. The HUDManager (responsible for user interface rendering) and the GameManager (responsible for win-condition evaluation) are permanently subscribed to this delegate. The GameManager assesses the event to determine if numerical parity has shifted, potentially triggering an endgame sequence. This strict separation ensures the networking logic never invokes UI rendering functions directly, strictly adhering to the Model-View-Controller (MVC) architectural

pattern.

3.3.3 The Task and System Interface Protocol

To manage the vast array of tasks and environmental interactables, the architecture utilizes a polymorphic approach. Base interfaces, such as ITask or ISystemType, define standard contracts encompassing initialization, progress reporting, and completion checks.
The ShipStatus object maintains arrays of these interface references. When a complex subsystem, such as the AutoDoorsSystem, requires state synchronization, it does not send individual booleans for every door. Instead, it serializes a 16-bit or 32-bit integer acting as a bitmask, where each bit corresponds to the open or closed state of a specific door. The receiving client's AutoDoorsSystem deserializes this bitmask, executing bitwise operations to update local states and trigger the respective door animation colliders simultaneously, vastly reducing serialization overhead.

3.4 Integration of Advanced Subsystems and Modding Architectures

The architectural reliance on a decentralized, client-driven simulation, combined with Unity's IL2CPP compiler, inadvertently fostered one of the most prolific modding and external integration ecosystems in modern software.

3.4.1 Spatial VoIP via Packet Interception

While the native application relies entirely on text-based communication, the architectural choice to compute spatial coordinates locally enables third-party VoIP integration with extreme efficiency. Tools such as *CrewComms* or *AmongUs-Mumble* operate externally by intercepting the unencrypted Unreliable UDP position packets directly from the local network interface, or by reading the memory addresses of the CustomNetworkTransform.
Because the PlayerControl component pairs a user's PlayerId with their spatial coordinates and life status (IsDead), external WebRTC or Discord RPC servers can calculate the 2D Euclidean distance between actors in real-time. This architecture dynamically modulates audio volume based on virtual proximity and strictly enforces communication silos—such as muting deceased players during active gameplay rounds or bridging communications between Impostors—without requiring any native codebase modifications.

3.4.2 BepInEx and the IL2CPP Hooking Paradigm

To achieve cross-platform optimization, Unity utilizes IL2CPP to convert C# intermediate language into highly optimized native C++ binaries. However, the structure of the InnerNetObject system is preserved within a metadata file (global-metadata.dat), exposing the architectural blueprint of the application.
Frameworks such as BepInEx leverage DLL search order hijacking or runtime injection to load custom Mono runtimes alongside the native application. By utilizing the preserved metadata, these frameworks identify the precise memory offsets of critical logical junctions, such as libil2cpp_base + 0xF3AF64, which corresponds to the PlayerControl.SetRole() method.
This allows community engineers to seamlessly inject highly complex asynchronous roles. For example, implementing a "Medic" or "Joker" role involves hooking into the SetRole() execution,

overriding the local faction boolean arrays, appending custom logical evaluation parameters into the GameManager's state loop, and injecting custom RPC byte codes (e.g., 0x30 CustomRoleAction) into the 0x05 GameData byte array. Because the Relay Server architecture is "dumb"—blindly forwarding Hazel reliable packets without strictly validating application-layer logic—modders can completely overwrite the foundational ruleset of the game dynamically, provided all connected clients run a mirrored injected library capable of deserializing the new RPC payloads.

4. Design Decisions and Technical Reasoning

The architecture of *Among Us* was not designed to support a massive, photorealistic physics simulation; it was aggressively engineered for a highly scalable, low-bandwidth, and hardware-agnostic mobile experience. Every major architectural paradigm reflects a distinct prioritization of bandwidth efficiency, infrastructure cost reduction, and gameplay fluidity over absolute cryptographic security or strict server-side physics authority.

4.1 Prioritization of RUDP via Hazel over TCP

The decision to utilize the Hazel framework for RUDP, rejecting standard TCP protocols (such as WebSockets) or bloated high-level engine networking solutions (like early iterations of Unity UNET or Mirror), is the most foundational design decision within the codebase.

Technical Reasoning: In an asynchronous multiplayer environment hosting 15 concurrent clients, transmitting spatial coordinate updates at a frequency of 10 to 20 ticks per second generates substantial packet volume. If TCP were utilized, the inherent design of the protocol would mandate that every packet is acknowledged sequentially. In the event of packet loss—a frequent occurrence on fluctuating 3G or 4G mobile networks—TCP enforces head-of-line blocking, stalling the entire incoming data stream until the dropped packet is retransmitted and received. Visually, this manifests as catastrophic "rubber-banding" and severe rendering stutters.

By implementing custom RUDP via Hazel, the architecture intelligently segments network traffic based on payload volatility. Spatial telemetry is dispatched via the Unreliable channel. If coordinate packet #45 is dropped, the local client simply ignores the gap, proceeding to interpolate the rendering transform toward packet #46 upon its arrival. This results in buttery-smooth rendering under poor network conditions. Conversely, critical commands like CastVote or MurderPlayer are dispatched via the Reliable channel, where Hazel attaches sequence identifiers and buffers the packets, executing micro-retransmissions independent of the spatial data stream until a 0x0a Acknowledgement is received. This dual-channel approach achieves the latency resilience of UDP without sacrificing the critical state integrity of TCP.

4.2 The Relay-Authoritative Hybrid Model

Unlike AAA competitive titles (such as *CS2* or *Apex Legends*) where a dedicated server calculates every ballistic trajectory, line-of-sight check, and physical collision to proactively neutralize cheating, this architecture relies on a partially authoritative relay model.

Technical Reasoning: Provisioning a dedicated server capable of running a full physical simulation of a Unity scene incurs significant CPU and Random Access Memory (RAM)

overhead. Given that the game scaled exponentially to hundreds of thousands of concurrent users globally, adopting a dedicated physics server model would have been financially ruinous for an independent studio.

By employing the Relay Server topology, the backend infrastructure operates purely as a routing matrix for byte[] arrays. The operational efficiency is staggering: an entry-level AWS nano instance can facilitate dozens of concurrent matches simultaneously, routing megabytes of data per day with negligible CPU utilization. The direct trade-off for this massive scalability is that the architecture inherently trusts the client application. The client autonomously calculates its Euclidean distance to a victim and independently informs the server via the MurderPlayer(TargetId) RPC that a kill has occurred.

While this architecture exposes the session to process memory-injection exploits—where modified clients spoof RPCs, teleport, or ignore logical cooldown timers—it allows the game to function viably. To mitigate this post-launch, developers introduced lightweight, heuristic anti-cheat checks on the relay server. Rather than simulating the environment, the server verifies basic logical constraints, tracking timestamps to enforce cooldowns and validating role assignments to prevent a Crewmate from successfully transmitting a MurderPlayer RPC, effectively neutralizing the most destructive spoofing attempts without compromising server efficiency.

4.3 Component-Based Networking (InnerNetObject)

The architectural choice to encapsulate network serialization logic into modular components (e.g., PlayerControl, PlayerPhysics, CustomNetworkTransform) rather than monolithically programming a singular, massive "NetworkedPlayer" class aligns with advanced software engineering principles regarding modularity and separation of concerns.

Technical Reasoning: This compositional approach grants extreme flexibility in entity generation and maintenance. A map entity like ShipStatus does not require velocity tracking or spatial interpolation, but it does mandate the serialization framework provided by the InnerNetObject to synchronize its child subsystems (like the ReactorSystem or LifeSuppSystem).

By decoupling the network transport layer from the specific application game logic, engineers can seamlessly introduce new mechanics or roles. For instance, when developers introduced the "Engineer" role—allowing a Crewmate to traverse the map via the Impostor's vent network—they did not need to rewrite the network serializer or the core physical systems. They simply updated the conditional logic within the PlayerPhysics class to accept the 0x13 EnterVent RPC if the client's internal role ID matched the Engineer enum. This modularity drastically reduces technical debt during iterative updates.

4.4 Aggressive Data Quantization and Compression

Throughout the network architecture, there is a systemic, rigorous effort to reduce the byte footprint of every payload transmitted over the Hazel protocol.

- Rather than transmitting strings for visual cosmetics or roles, the system serializes single-byte enum IDs (e.g., HatID.Plague, ColorID.Black).
- Lobby identification codes are algorithmically packed from six alphanumeric characters

into a single 32-bit integer.

- The ShipStatus class synchronizes vast arrays of localized task completions and environmental door states using compact bitwise flags rather than bloated arrays of booleans.

Technical Reasoning: During peak operational periods, database latency and server egress bandwidth costs become the primary bottleneck for multiplayer infrastructure. An unoptimized packet structure, multiplying across 15 concurrent clients and transmitting up to 20 spatial updates per second, scales exponentially and rapidly exhausts network capacities. By aggressively utilizing bit-packing, variable-length integer encoding (PackedUInt32), and delta compression, the architecture maintains an astonishingly low payload footprint, achieving synchronization with approximately 0.5 to 0.57 kilobits per second per player. This extreme optimization allows the application to function flawlessly even on heavily degraded 2G or 3G mobile networks, a fundamental architectural factor contributing to its unprecedented global market penetration across diverse socio-economic hardware profiles.

The code architecture of *Among Us* ultimately represents a masterclass in pragmatic, scalable network design. By eschewing computationally expensive dedicated physics servers in favor of a highly optimized UDP relay topology backed by the reliable sequencing of the Hazel framework, the architecture achieves a frictionless, cross-platform experience. The strict separation of concerns—where quantized byte streams are translated into modular InnerNetObject RPCs, which subsequently update decoupled physical and presentation systems—creates a resilient codebase capable of supporting ongoing feature expansion, complex asymmetrical roles, and an expansive modding ecosystem.